Deux modèles, deux mesures

Ewen DUFOUR

Encadré par
Thibaut BALABONSKI

mars - juillet 2025

Résumé

Au cours de ce travail, nous définissons une mesure d'espace sur un modèle dérivé du λ-calcul. Dans un premier temps, nous montrons que ce modèle peut être simulé par des machines de Turing avec une consommation d'espace cohérente, en passant par une machine abstraite. Dans un second temps, nous implémentons un langage de scripts permettant la manipulation de termes ainsi que l'utilisation de machines abstraites sur ceux-ci.

1 Introduction

Thèse d'invariance des modèles. Énoncée par Peter van Emde Boas et Cees Slot [6], la thèse d'invariance des modèles affirme qu'il existe une classe de modèles de calcul et que tous les modèles au sein de cette classe peuvent se simuler entre eux avec un surcoût polynomial en temps et linéaire en espace. En particulier, cette classe contiendrait les machines de Turing, les RAM, etc.

Le λ-calcul et sa complexité. Le λ-calcul est un modèle de calcul servant de base aux langages de programmation fonctionnels. Il est donc naturel de nous assurer que ce modèle est bien dans cette classe. Bien qu'étant naturelles sur les machines de Turing, les notions de temps et d'espace sur les λ-termes et leurs réductions ne sont pas immédiates. Dans ce travail, nous étudierons la complexité du λ-calcul à travers le Linear Substitution Calculus.

Le linear substitution calculus. Le Linear Substitution Calculus est une variante du λ-calcul introduite par Accattoli et co-auteurs [1]. Les termes sont donnés par la grammaire suivante :

$$t ::= \ x \mid \lambda x.t \mid t\ t \mid t[x/t]$$

En plus des termes habituels du λ-calcul, nous ajoutons une construction $t[x/t']$ qui explicite la substitution de x par t' dans t. Nous ajoutons à ces termes les règles de réduction suivantes :

$$\begin{array}{rcll} \mathcal{L}[\lambda x.t]\ u & \to_{d\beta} & \mathcal{L}[t[x/u]] & \\ \mathcal{C}[\![x]\!][x/u] & \to_{ls} & \mathcal{C}[\![u]\!][x/u] & \\ t[x/u] & \to_{gc} & t & \qquad x \notin fv(t) \end{array}$$

où $\mathcal{C}$ et $\mathcal{L}$ sont des contextes construits par les grammaires :

$$\mathcal{L} ::= \square \mid \mathcal{L}[x/t]$$

$$\mathcal{C} ::= \square \mid \mathcal{C}[x/t] \mid t[x/\mathcal{C}] \mid \mathcal{C}\ t \mid t\ \mathcal{C} \mid \lambda x.\mathcal{C}$$

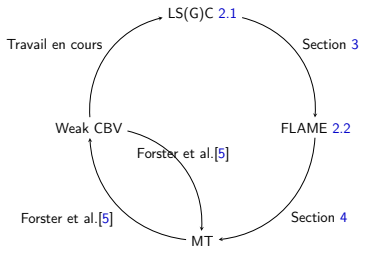

FIGURE 1 – Plan de simulation des modèles.

Travaux connexes. Comme mentionnée précédemment, la question de la complexité peut se décliner sous deux formes : spatiale et temporelle. Concernant la complexité en temps, Dal Lago et Martini montrent que le nombre de β-réduction est une mesure cohérente pour le λ-calcul faible [4], c'est-à-dire sans réduction sous les λ-abstractions. Accattoli et Dal Lago montrent finalement que cette mesure reste cohérente en autorisant les réductions sous les abstractions[2].

Bien que la question du temps soit réglée, celle de l'espace ne possède toujours pas de réponse pleinement satisfaisante. Dans leurs travaux, [5], Forster et al. montrent que le λ-calcul en appel par valeur est raisonnable pour l'espace (et le temps). Cependant, leur mesure ne permet pas de capturer les classes sous-linéaires. En revanche, Accattoli, Dal Lago et Vanoni décrivent une manière permettant de capturer ces classes dans leurs travaux [3]. Néanmoins, leurs mesures se basent sur une machine abstraite et non pas sur les λ-termes.

Contributions. Le schéma en figure 1 montre les différentes étapes que nous suivons pour montrer que notre modèle de calcul et sa mesure sont cohérents. Dans ce travail, nous commençons par définir LS(G)C et FLAME, les modèles que nous allons étudier, ainsi que leurs mesures d'espaces respectifs. Par la suite, nous montrons que FLAME simule LS(G)C avec un surcoût d'espace linéaire. Nous continuons en montrant que FLAME peut être simulée par une machine de Turing en utilisant également un surcoût d'espace linéaire. Nous finissons par implémenter un langage permettant de décrire des termes et de les fournir à des machines abstraites.

2 Formalismes utilisés

2.1 LS(G)C

2.1.1 Stratégie de réduction

Pour cette stratégie de réduction, nous distinguons une classe de termes que nous appellerons des valeurs. Ceux-ci sont donnés par la grammaire :

$$v ::= \lambda x.t \mid v[x/t]$$

Les valeurs sont donc les abstractions munies d'une « liste » de substitutions. Nous distinguons clairement ici le concept de fermeture. De plus, nous restreignons les contextes que nous utiliserons dans nos réductions à ceux donnés par la grammaire :

$$\mathcal{C} ::= \Box \mid \mathcal{C}[x/t] \mid \mathcal{C}\ t \mid v\ \mathcal{C}$$

Nous remarquons que l'on ne permet plus la substitution sous les abstractions ni dans les substitutions explicites. Notre stratégie est donc définie ainsi :

$$\frac{t \to t'}{t\ u \to t'\ u}\ LA \qquad \frac{t \to t'}{v\ t \to v\ t'}\ RA$$

$$\frac{t \to t' \quad x \notin fv(t')}{t[x/u] \to t'}\ GC \qquad \frac{t \to t' \quad x \in fv(t')}{t[x/u] \to t'[x/u]}\ US$$

$$\frac{x \in fv(\mathcal{C}[\![u]\!])}{\mathcal{C}[\![x]\!][x/u] \to \mathcal{C}[\![u]\!][x/u]}\ LS \qquad \frac{x \notin fv(\mathcal{C}[\![u]\!])}{\mathcal{C}[\![x]\!][x/u] \to \mathcal{C}[\![u]\!]}\ LSGC$$

$$\frac{x \in fv(t)}{\mathcal{L}[\lambda x.t]\ v \to \mathcal{L}[t[x/v]]}\ DB \qquad \frac{x \notin fv(t)}{\mathcal{L}[\lambda x.t]\ v \to \mathcal{L}[t]}\ DBGC$$

FIGURE 2 – Règles de réduction du LS(G)C

Explications. Les règles RA et LA sont la base de l'appel par valeur. Nous commençons par évaluer le terme de gauche d'une application jusqu'à obtenir une abstraction avec ou sans environnement, puis on évalue le terme de droite jusqu'à obtenir une valeur. Ensuite, les règles DB et $DBGC$ interviennent. Leur rôle est d'effectuer l'application et de ne garder la substitution explicite que si elle est utile ($DBGC$ étant la composition de $\to_{d\beta}$ et $\to_{gc}$). Les règles LS et $LSGC$ servent à effectuer une substitution lorsque l'on évalue une variable et à supprimer la substitution explicite si elle devient inutile. Enfin, les règles GC et US vont aller évaluer les termes sous les substitutions explicites et les supprimer si elles deviennent inutiles après l'évaluation du sous-terme. Cela permet de propager le « gc » et de ne garder que les substitutions utiles.

Nous considérons que les termes sont représentés en utilisant des indices de de Bruijn. Cependant, les indices peuvent faire référence soit aux abstractions, soit aux substitutions explicites. Voici deux exemples de termes avec cette notation :

$$(\lambda.0\ 1)[\lambda.0] = (\lambda x.x\ y)[y/\lambda z.z]$$
$$\lambda.(0\ 1)[\lambda.0] = \lambda x.(y\ x)[y/\lambda z.z]$$

De plus, les termes au sein des substitutions explicites peuvent eux-mêmes faire référence à d'autres substitutions. Il nous faut donc faire attention à bien gérer les indices lors des DB, $DBGC$, LS, $LSGC$ et GC.

2.1.2 Mesure d'espace pris par les termes

Nous définissons la mesure d'espace ainsi :

$$|x| = |dB(x)|$$
$$|\lambda x.t| = 1 + |t|$$
$$|t\ u| = |t| + |u|$$
$$|t[x/u]| = |t| + |u|$$

L'utilisation des indices de de Bruijn nous permet de ne pas avoir à stocker les variables des substitutions explicites, ce qui explique leurs coûts. Nous étendons naturellement cette définition aux contextes.

2.2 La machine FLAME

FLAME est une machine abstraite qui implante la stratégie d'appel par valeur pour les termes du LSC. Un état de FLAME se décrit par un triplet annoté de la forme $\langle (v, n), t, \mathcal{C} \rangle_e$ où :

— t représente le terme en train d'être évalué.
— $\mathcal{C}$ est une pile de contexte d'évaluation.
— (v, n) contient optionnellement l'argument d'une application et sa « distance » par rapport à l'abstraction.
— e est un nombre qui indique la phase de fonctionnement de FLAME.

Toutes les transitions de FLAME sont décrites dans la figure 7 en annexe.

2.2.1 Fonctionnement

Définition 2.1 (Décomposition d'un contexte)**.** La décomposition d'un contexte correspond à la liste des parties « atomiques » qui le composent.

$$\begin{aligned} d(\square) &= \square \\ d(\mathcal{C}[x/v]) &= d(\mathcal{C}) :: \square[x/v] \\ d(\mathcal{C}\ t) &= d(\mathcal{C}) :: \square\ t \\ d(v\ \mathcal{C}) &= d(\mathcal{C}) :: v\ \square \end{aligned}$$

Cette définition s'étend naturellement à tout type de contexte.

L'évaluation des termes se fait en trois phases : la décomposition, la réduction et la reconstruction.

Décomposition. Dans le premier temps, la machine décompose les termes jusqu'à trouver une réduction possible (substitution ou β-réduction). La machine sépare les applications en évaluant un des membres et stocke l'autre sur sa pile de contextes. De même, elle stocke sur sa pile les substitutions et évalue leur sous-terme gauche (si elle ne fait pas partie d'une valeur). Nous voyons immédiatement que cette phase implémente la fonction de décomposition d'un contexte (2.1). Cependant, elle ne stocke pas les contextes vides sur la pile. Cette phase d'exploration est réalisée par les transitions dont l'état de départ est 0.

Réduction. Lorsque la machine a trouvé une réduction, elle va effectuer une transition $\rightsquigarrow_{db}$ et rentrer dans l'état 1 si c'est une β-réduction, ou effectuer une transition $\rightsquigarrow_{ls}$ et entrer dans l'état 2 si c'est une substitution. La machine se sert de l'état 1 pour réaliser une réduction du genre $d\beta$, il est utilisé pour calculer le décalage des indices de de Bruijn et mettre la nouvelle substitution sous les potentielles substitutions déjà présentes. La machine entre ensuite dans l'état 2.

Reconstruction. Après avoir effectué une réduction, la machine va reconstruire le terme. Lors de cette phase, elle récupère les sous-termes de sa pile de contexte et reconstruit le terme. Lorsqu'elle trouve une substitution dans sa pile de contexte, elle vérifie que la substitution est bien liée dans son terme. Si ce n'est pas le cas, elle supprime cette substitution lors de la transition $\rightsquigarrow_{gc}$. Dans le cas contraire, elle replace simplement la substitution sur son terme courant.

Relance. Lorsque la machine a fini de reconstruire un terme, si celui-ci est une valeur, elle entre dans l'état 3 et s'arrête complètement. Dans le cas contraire, la machine retourne dans l'état 0 et effectue un nouveau cycle d'évaluation.

2.2.2 Mesure d'espace

Afin d'effectuer nos analyses sur l'espace pris par notre machine, nous définissons la mesure suivante :

$$|\langle (v,n), t, \mathcal{C}\rangle_e| = |t| + |v| + |n| + |\mathcal{C}|$$

Dans le cas où l'entrée **arg** de la machine est **-**, nous prenons comme taille 0. L'entrée **état** de la machine ne pouvant être que **0**,**1**,**2** ou **3** nous ne comptons pas cette information dans nos calculs.

3 Simulation de LS(G)C

3.1 Propreté de LS(G)C

Étant donné un terme t, nous disons de ce terme qu'il est propre s'il ne possède aucune substitution explicite inutile. Cela nous assure que nous ne pourrons pas effectuer de réduction GC sur le terme t.

Définition 3.1 (Propreté)**.** Un terme t est dit propre si $cl(t)$ possède une dérivation.

$$\frac{}{cl(x)} \qquad \frac{cl(t) \quad cl(u)}{cl(t\ u)} \qquad\qquad \frac{cl(t)}{cl(\lambda x.t)} \qquad \frac{cl(t) \quad cl(u) \quad x \in fv(t)}{cl(t[x/u])}$$

FIGURE 3 – Règles de dérivation de la propreté.

Théorème 3.1 (Préservation de la propreté)**.** Soient t_0 et t_1 des termes.

$$t_0 \text{ propre } \wedge\ t_0 \to\ t_1\ \Rightarrow\ t_1 \text{ propre}$$

Démonstration. Par induction sur la dérivation de $t_0 \to t_1$. □

Ce théorème nous assure donc que nos règles de calcul ne produisent aucun terme avec des substitutions explicites pouvant être nettoyées.

3.2 Comportement de FLAME

Afin de montrer que FLAME simule raisonnablement LS(G)C, nous devons nous intéresser à ses différentes phases.

Dans un premier temps, nous voulons que la machine décompose correctement les termes en n'utilisant aucun espace superflu.

Lemme 3.2 (Décomposition)**.** Soient $\mathcal{C}$, $\mathcal{D}$ des contextes d'évaluation et t un terme.

$$\langle -, \mathcal{C}[t], \mathcal{D}\rangle_0 \rightsquigarrow^* \langle -, t, \mathcal{C} :: \mathcal{D}\rangle_0$$

en utilisant un espace $|\mathcal{C}| + |t| + |\mathcal{D}|$

Démonstration. Par induction sur la forme de $\mathcal{C}$. □

Dans un second temps, il nous faut nous assurer que la machine effectue correctement les réductions et quantifier l'espace pris lors de cette phase.

Lemme 3.3 (Plongeon sous les substitutions)**.** Soient $\mathcal{D}$ un contexte d'évaluation, $\mathcal{L}$ un contexte de substitution et t, u des termes.

$$\langle (u, n), \mathcal{L}[t], \mathcal{D}\rangle_1 \rightsquigarrow^k_{rdb} \langle (u, n+k), t, \mathcal{L} :: \mathcal{D}\rangle_1$$

où k est la longueur de la décomposition de $\mathcal{L}$, en utilisant un espace $|\mathcal{D}| + |t| + |\mathcal{L}| + |u| + |n+k|$

Démonstration. Par induction sur la forme de $\mathcal{L}$. □

Finalement, nous souhaitons que la machine reconstruise les termes correctement dans un espace correspondant à la taille de ceux-ci.

Lemme 3.4 (Reconstruction)**.** Soient $\mathcal{C}$ et $\mathcal{D}$ des contextes d'évaluation et t un terme.

$$\mathcal{C}[t] \text{ propre } \Rightarrow \langle -, t, \mathcal{C} :: \mathcal{D}\rangle_2 \rightsquigarrow^* \langle -, \mathcal{C}[t], \mathcal{D}\rangle_2$$

en utilisant un espace $|\mathcal{C}| + |t| + |\mathcal{D}|$

Démonstration. Par induction sur la forme de $\mathcal{C}$. □

Grâce au lemme de décomposition 3.2, nous nous assurons que la machine effectue correctement la décomposition des contextes et quantifions l'espace pris par cette opération. Le lemme de plongeon sous les substitutions 3.3 nous assure quant à lui que l'argument d'une application est bien amené devant les substitutions explicites lors d'une « transition dβ ». De plus, nous voyons que le décalage à effectuer sur les indices de de Bruijn est bien calculé et que nous donnons l'espace utilisé par cette phase. Finalement, le lemme de reconstruction 3.4 nous montre que le terme est correctement reconstruit après son évaluation et nous donne également l'espace pris par la machine pour cette opération.

3.3 Simulation raisonnable de LS(G)C par FLAME

Grâce aux lemmes que nous venons de décrire, nous pouvons donc montrer que FLAME simule raisonnablement LS(G)C.

Théorème 3.5 (Simulation raisonnable d'un pas)**.** Soient t et u des termes propres et $\mathcal{D}$ un contexte d'évaluation. Il existe une constante k strictement positive telle que

$$\begin{gathered}\text{Si } t \to u \text{ avec un espace } S\text{, alors}\\ \langle -, t, \mathcal{D}\rangle_0 \rightsquigarrow^* \langle -, u, \mathcal{D}\rangle_2 \text{ avec un espace } k \times S + |\mathcal{D}|\end{gathered}$$

Démonstration. Par induction sur la dérivation de $t \to u$. □

Et par applications successives du théorème, nous obtenons le corollaire suivant :

Corollaire 3.6. Soient t et u des termes. Il existe une constante k strictement positive telle que

$$\begin{gathered}\text{Si } t \to^* u \text{ avec } u \text{ une valeur en espace } S\text{, alors}\\ \langle -, t, \square\rangle_0 \rightsquigarrow^* \langle -, u, \square\rangle_3 \text{ avec un espace } k \times S\end{gathered}$$

Donc FLAME simule bien LS(G)C avec un surcoût d'espace linéaire.

4 Simulation de FLAME

Montrons maintenant que FLAME peut être simulée par des machines de Turing avec un espace raisonnable. Pour ce faire, il nous faut trois choses :

— Un encodage des termes ayant un surcoût d'espace linéaire par rapport à la taille de nos termes.
— Une représentation des différentes entrées de la machine.
— Les fonctions internes de FLAME **red**, **lift**, **isFree** et **isValue**. Ces fonctions vont respectivement réduire ou augmenter les indices de de Bruijn dans un terme encodé, vérifier qu'une variable est libre dans un terme encodé et vérifier qu'un terme encodé est une valeur. De nouveau, ces fonctions doivent se faire avec un coût spatial au plus linéaire.

Encodage. L'encodage des termes dans se fait via une fonction récursive des termes dans l'alphabet $\{0,\ 1,\ \lambda,\ A,\ \text{␣},\ (,\ ),\ S,\ [,\ ]\}$ définie ainsi :

$$\begin{aligned}
\mathcal{E}(x) &= x \\
\mathcal{E}(\lambda.t) &= \lambda\mathcal{E}(t)A \\
\mathcal{E}(t\ u) &= (\mathcal{E}(t)\text{␣}\mathcal{E}(u)) \\
\mathcal{E}(t[v]) &= S\mathcal{E}(t)[\mathcal{E}(v)]
\end{aligned}$$

Nous partons du principe que les termes avant encodage sont sous la forme d'indice de de Bruijn et les variables sont écrites en base 2. Nous avons ajouté un symbole A pour délimiter la fin d'une abstraction et un symbole S pour définir le début d'une substitution explicite. Enfin, nous ajoutons des parenthèses et un symbole d'espace aux applications pour délimiter chaque membre. Il est direct que notre encodage a une taille linéaire en la taille du terme non encodé. Par la suite, nous noterons $\bar{t}$ l'encodage du terme t.

Choix du modèle de calcul. Pour représenter les différentes entrées de la machine, nous allons utiliser une machine à quatre rubans. Pour les arguments, nous écrivons l'encodage du terme suivi par le nombre n (la « distance à l'abstraction) sur le premier ruban. Les termes sont décrits par leur encodage sur le deuxième ruban. Nous sauvegardons les contextes sur le troisième ruban où nous écrivons les encodages des sous-termes précédés par des symboles qui permettent de différencier les types de contexte (LS, RA et LA dans FLAME). Sur le dernier ruban, nous gardons en mémoire l'état de la machine et écrivons les résultats des fonctions **isFree** et **isValue**. Ce choix de modèle fonctionne, car les machines de Turing à plusieurs rubans peuvent être simulées par des machines à un ruban avec un espace linéaire (Théorème 1 des travaux de van Emde Boas [7]).

4.1 Fonctions internes de FLAME

Maintenant, voyons comment effectuer les différentes opérations que fait la machine sur notre encodage.

isValue. Il suffit de parcourir l'encodage de gauche à droite. Si nous rencontrons sur le symbole S, nous continuons. Si le symbole rencontré est λ renvoyer $vrai$, sinon renvoyer $faux$. Tout cela se fait en espace constant.

isFree. Afin de décider si une variable est libre, il suffit de parcourir une unique fois l'encodage en comptant le nombre de lieurs rencontrés. L'algorithme n'accédant à l'encodage qu'en lecture, il n'a besoin de stocker que le nombre de lieurs. Il utilise donc un espace logarithmique en la longueur de l'encodage. Le programme en figure 6 en annexe est une implémentation dans le langage OCaml de cette fonction.

lift. Pour augmenter les indices de de Bruijn dans un encodage reçu en entrée, une machine de Turing doit parcourir son entrée une unique fois. Elle écrit sur sa bande de renvoi l'encodage en augmentant à la volée les variables libres. Ainsi, l'espace utilisé par la machine est celui utilisé pour détecter qu'une variable est libre ainsi que celui pour additionner deux nombres. La fonction s'exécute ainsi dans un espace linéaire en la longueur de l'encodage. La figure 7 en annexe montre une fonction OCaml permettant de calculer ce décalage. Cependant, cette fonction, bien qu'utilisant un espace linéaire en la taille de l'encodage, utilise plus que l'espace décrit précédemment, car elle doit stocker temporairement l'encodage augmenté.

red. Un procédé analogue nous permet de calculer la réduction d'un encodage avec un coût d'espace linéaire.

Les descriptions ci-dessus permettent donc de justifier l'existence d'une machine de Turing simulant FLAME avec un surcoût d'espace linéaire.

Théorème 4.1. Il existe une machine de Turing $\mathcal{M}$ et une constante k telles que si $\langle -, t, \square \rangle_0 \rightsquigarrow^* \langle -, u, \square \rangle_3$ avec un espace S, alors $\mathcal{M}\ \bar{t} \to^*\ \bar{u}$ avec un espace $k \times S$.

FLAME peut être simulée par les machines de Turing avec un surcoût d'espace linéaire. Finalement, LS(G)C peut être simulé raisonnablement (pour l'espace) par les machines de Turing.

Théorème 4.2 (Simulation)**.** Soient t et u des termes. Il existe une machine de Turing $\mathcal{M}$ et une constante k telles que si $t \to^* u$ avec u une valeur en espace S, alors $\mathcal{M}\ \bar{t} \to^*\ \bar{u}$ avec un espace $k \times S$.

Démonstration. En utilisant le corollaire 3.6 et le theorème 4.1. □

5 Implémentation

Nous souhaitons étudier le comportement de FLAME, nous assurer qu'elle évalue correctement les termes ou encore voir les séquences d'évaluation des termes. C'est dans ce but que nous développons Flash, un langage de script. Ce langage nous permet de déclarer des termes, ceux-ci sont traduits et encodés dans un format spécifié par l'utilisateur. Il permet ensuite de fournir aux machines abstraites les termes et de récupérer leurs résultats.

Machines abstraites. Nous implémentons deux machines abstraites dans nos travaux. Premièrement, nous implémentons FLAME. Cette machine travaille sur des lsc-termes sous leur forme de de Bruijn. Son implémentation se trouve en figure 8 en annexe. Secondement, nous implémentons la Space-KAM [3], celle-ci prend en entrée et renvoie des λ-termes nommés. Les deux machines ne fonctionnant pas avec des termes de même nature, nous devons être capables de déclarer des termes sous différents formats.

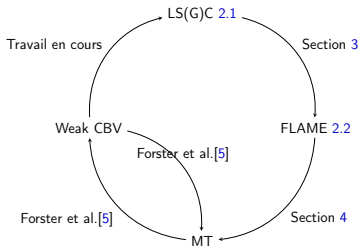

FIGURE 4 – Rappel des contributions.

Déclaration et encodage des termes. Comme nous l'avons vu, nos machines travaillent sur des termes de nature différente. Nous utilisons donc des mots clés devant nos déclarations afin de spécifier l'encodage des termes. Actuellement, nous pouvons déclarer des termes avec les mots clés **lsc** et **lambda**. Chaque mot-clé permet de définir des termes en utilisant des constructions qui peuvent leur être propres (ex. t[x/v] avec lsc). Les déclarations de termes se font en utilisant des variables nommées et non en indices de de Bruijn. Lorsqu'un terme lsc est déclaré, Flash traduit la forme nommée vers une forme de de Bruijn tout en gardant une trace du nom des lieurs. Cela nous permet de garder une forme plus agréable pour l'utilisateur lors de l'affichage d'un terme. Maintenant que nous pouvons déclarer nos termes et que nous avons nos machines, il faut que notre langage fasse interagir ces entités.

Structure d'un programme Flash. Un programme Flash est une séquence de scripts séparés par « ; ; ». Chaque script est composé de deux choses : des définitions de termes et des expressions à évaluer. Les déclarations et expressions d'un même script sont séparées par « ; ». Un terme n'est accessible que dans le script dans lequel il est déclaré. Les expressions d'un script sont des suites d'applications qui peuvent soit prendre la forme $e_1\ e_2$ (on applique e_1 à e_2), ou bien la forme $e_2| > e_1$ (on passe en argument e_2 à e_1). Elles permettent, entre autres, de fournir aux machines les termes déclarés, d'afficher des termes, calculer leur taille, etc. Un exemple de programme Flash et son résultat se trouve sur la figure 9.

Ce langage possède les avantages d'être minimaliste et très modulable. En effet, nous avons la possibilité d'interagir avec des termes et des machines complètement distincts. Pour implémenter et communiquer avec une nouvelle machine, il suffit de définir un encodage et des constructions pour les termes, ainsi que d'implémenter une machine abstraite.

6 Conclusion et perspectives

Nous avons commencé ce travail par définir notre modèle de départ, LS(G)C ainsi que sa mesure d'espace. De même pour notre modèle intermédiaire, FLAME. Par la suite, nous avons montré que FLAME simulait LS(G)C avec un espace raisonnable, et qu'elle était elle-même simulable par une machine de Turing avec un surcoût d'espace linéaire. Nous avons terminé en implémentant un petit langage de script permettant de définir des termes et de les donner à des machines abstraites.

Pour continuer nos travaux, la première chose à faire serait de montrer que LS(G)C peut simuler les machines de Turing avec un surcoût d'espace linéaire également. Une des pistes envisageables serait de se baser sur les travaux de Forster et al. [5] et de montrer que LS(G)C simule le λ-calcul en appel par valeur faible. Cependant, nos mesures ne semblent pas permettre de capturer les classes sous-linéaires. Nous pourrions, pour cela, essayer d'étendre le travail de Accattoli, Dal Lago et Vanoni [3] en proposant un modèle de calcul qui se rapprocherait plus du

λ-calcul et serait moins bas niveau que la Space-KAM.

7 Remerciements

Je remercie très chaleureusement Thibaut pour avoir encadré ce stage ainsi que pour toute l'aide qu'il m'a apporté. Je tiens également à remercier Travis pour son aide durant ce stage ainsi que l'équipe du LMF qui s'est montrée accueillante et très sympathique.

Références

[1] B. Accattoli, E. Bonelli, D. Kesner, and C. Lombardi. A nonstandard standardization theorem. *ACM SIGPLAN Notices*, 49(1) :659–670, 2014.

[2] B. Accattoli and U. Dal Lago. (leftmost-outermost) beta reduction is invariant, indeed. *Logical Methods in Computer Science*, 12, 2016.

[3] B. Accattoli, U. Dal Lago, and G. Vanoni. Reasonable space for the λ-calculus, logarithmically. In *Proceedings of the 37th Annual ACM/IEEE Symposium on Logic in Computer Science*, pages 1–13, 2022.

[4] U. Dal Lago and S. Martini. The weak lambda calculus as a reasonable machine. *Theoretical Computer Science*, 398(1-3) :32–50, 2008.

[5] Y. Forster, F. Kunze, and M. Roth. The weak call-by-value λ-calculus is reasonable for both time and space. *Proceedings of the ACM on Programming Languages*, 4(POPL) :1–23, 2019.

[6] C. Slot and P. van Emde Boas. The problem of space invariance for sequential machines. *Information and Computation*, 77(2) :93–122, 1988.

[7] P. van Emde Boas. Machine models and simulations. 1987.

Annexe

arg	**terme**	**contexte**	**état**	**transition**	**arg**	**terme**	**contexte**	**état**	
–	$t[u]$	$\mathcal{C}$	0	$\rightsquigarrow_{us}$	–	t	$\mathsf{LS}(u) :: \mathcal{C}$	0	
-	$t\ u$	$\mathcal{C}$	0	$\rightsquigarrow_{lapp}$	–	t	$\mathsf{RA}(u) :: \mathcal{C}$	0	
-	$v\ t$	$\mathcal{C}$	0	$\rightsquigarrow_{rapp}$	–	t	$\mathsf{LA}(v) :: \mathcal{C}$	0	
-	n	$\mathcal{C}$	0	$\rightsquigarrow_{ls}$	-	$\uparrow^{n+1} t$	$\mathcal{C}$	2	$\mathsf{LS}(t) = \mathsf{nth_subst}(\mathcal{C})$
-	$v_1\ v_2$	$\mathcal{C}$	0	$\rightsquigarrow_{db}$	$(v_2, 0)$	v_1	$\mathcal{C}$	1	
(v_2, n)	$v_1[t]$	$\mathcal{C}$	1	$\rightsquigarrow_{rdb}$	$(v_2, n+1)$	v_1	$\mathsf{LS}(t) :: \mathcal{C}$	1	
(v, n)	$\lambda.t$	$\mathcal{C}$	1	$\rightsquigarrow_{fdb}$	-	t	$\mathsf{LS}(\uparrow^n v) :: \mathcal{C}$	2	$0 \in fv(t)$
(v, n)	$\lambda.t$	$\mathcal{C}$	1	$\rightsquigarrow_{fdbgc}$	-	$\downarrow t$	$\mathcal{C}$	2	$0 \notin fv(t)$
-	v	$[]$	2	$\rightsquigarrow_{stop}$	-	v	$[]$	3	
-	t	$[]$	2	$\rightsquigarrow_{dive}$	-	t	$[]$	0	
-	t	$\mathsf{LS}(u) :: \mathcal{C}$	2	$\rightsquigarrow_{gc}$	-	$\downarrow t$	$\mathcal{C}$	2	$0 \notin \mathsf{fv}(t)$
-	t	$\mathsf{LS}(u) :: \mathcal{C}$	2	$\rightsquigarrow_{bsubst}$	-	$t[u]$	$\mathcal{C}$	2	$0 \in \mathsf{fv}(t)$
-	t	$\mathsf{RA}(u) :: \mathcal{C}$	2	$\rightsquigarrow_{blapp}$	-	$t\ u$	$\mathcal{C}$	2	
-	t	$\mathsf{LA}(u) :: \mathcal{C}$	2	$\rightsquigarrow_{brapp}$	-	$u\ t$	$\mathcal{C}$	2	

FIGURE 5 – Transitions de FLAME.

```
let rec free (n : int) (e : encoding) : bool =
  if n < 0 then false
  else
    match e with
    | [] -> false
    | BeginLambda :: el -> free (n + 1) el
    | EndLambda :: el -> free (n - 1) el
    | BeginSubst :: el -> free (n + 1) el
    | (LeftPar  | RightPar  | RightBrack | Space) :: el ->
        free n el

    | LeftBrack :: el -> free (n - 1) el
    | Num i :: el -> if i = n then true else free n el
```

FIGURE 6 – Fonction OCaml décidant si une variable est libre.

```
let lift_encoding shift e =
  let rec aux shift bound_nb acc = function
    | [] -> List.rev acc
    | Num n :: es when n < bound_nb -> aux shift bound_nb (Num n :: acc
      ) es
    | Num n :: es -> aux shift bound_nb (Num (n + shift) :: acc) es
    | BeginLambda :: es -> aux shift (bound_nb + 1) (BeginLambda :: acc
      ) es
    | BeginSubst :: es -> aux shift (bound_nb + 1) (BeginSubst :: acc)
      es
    | LeftBrack :: es -> aux shift (bound_nb - 1) (LeftBrack :: acc) es
    | RightBrack :: es -> aux shift bound_nb (RightBrack :: acc) es
    | Space :: es -> aux shift bound_nb (Space :: acc) es
    | LeftPar :: es -> aux shift bound_nb (LeftPar :: acc) es
    | RightPar :: es -> aux shift bound_nb (RightPar :: acc) es
    | EndLambda :: es -> aux shift (bound_nb - 1) (EndLambda :: acc) es
  in
  aux shift 0 [] e
```

FIGURE 7 – Fonction OCaml augmentant les indices dans un encodage.

```
let flame t =
  (* Meta operations for de Bruijn indices*)
  ...
  (* Size computation *)
  let maxi = ref 0 in

  (* Reconstruction *)
  let rec build_lsc (conts : context list) (t : lsc) : lsc =
    match conts with
    | LA u :: cl -> build_lsc cl (App (u, t))
    | RA u :: cl -> build_lsc cl (App (t, u))
    | LS (s, u) :: cl when is_free 0 t -> build_lsc cl (Ls (s, t, u))
    | LS _ :: cl -> build_lsc cl (red t)
    | [] when is_clean_value t ->
        maxi := max !maxi (size t);
        t
    | [] -> dive t

  (* Reduction *)
  and db conts u v =
    let rec db_aux conts u v prof =
      match u with
      | Abs (o, t') -> build_lsc (LS (o, lift prof v) :: conts) t'
      | Ls (o, t', u') -> db_aux (LS (o, u') :: conts) t' v (prof + 1)
      | _ -> assert false
    in
    db_aux conts u v 0

  (*Decomposition*)
  and dive t =
    PP.print_lsc_nl t;
    maxi := max !maxi (size t);
    let rec plonge (conts : context list) (te : lsc) : lsc =
      match te with
      | Ls (o, t, u) -> plonge (LS (o, u) :: conts) t
      | Var n ->
          let to_plug = lift (n + 1) (find_subst conts n) in
          maxi := max !maxi (cl_size conts + size to_plug);
          build_lsc conts to_plug
          (* Ls *)
      | App (t, u) when is_clean_value t && is_clean_value u ->
          db conts t u (* Db *)
      | App (t, u) when is_clean_value t && not (is_clean_value u) ->
          plonge (LA t :: conts) u
      | App (t, u) -> plonge (RA u :: conts) t
      | t -> t
    in
    plonge [] t
  in
  let ((_, t) as res) = (!maxi, dive t) in
  PP.print_lsc_nl t;
  res
```

FIGURE 8 – Implémentation de FLAME en OCaml.

```
lambda t1 = fun x -> x ;
lambda t2 = fun y -> t1 y;
lambda t3 = fun t1 -> t2 t1;
"Spacekam:" |> print;
t2 t1 |> spacekam ;;


lsc t1 = fun x -> x ;
lsc t2 = fun y -> t1 y;
lsc t3 = fun t1 -> t2 t1;
"Flame:\n"|> print;
t2 t1 |> flame
```

```
Flash version 2.0.0

Script 0:
Spacekam:

{term: (λy.(λx.x) y) (λx.x)
env: []
stack: []}

{term: λy.(λx.x) y
env: []
stack: (λx.x, [])}

{term: (λx.x) y
env: [y ← (λx.x, [])]
stack: []}

{term: λx.x
env: []
stack: (λx.x, [])}

{term: x
env: [x ← (λx.x, [])]
stack: []}

{term: λx.x
env: []
stack: []}


Script 1:
Flame:

(λy.(λx.x) y) (λx.x)
((λx.x) y)[y/λx.x]
(λx.x) (λx.x)
x[x/λx.x]
λx.x
```

FIGURE 9 – Exemple de code flash et résultat d'éxecution.